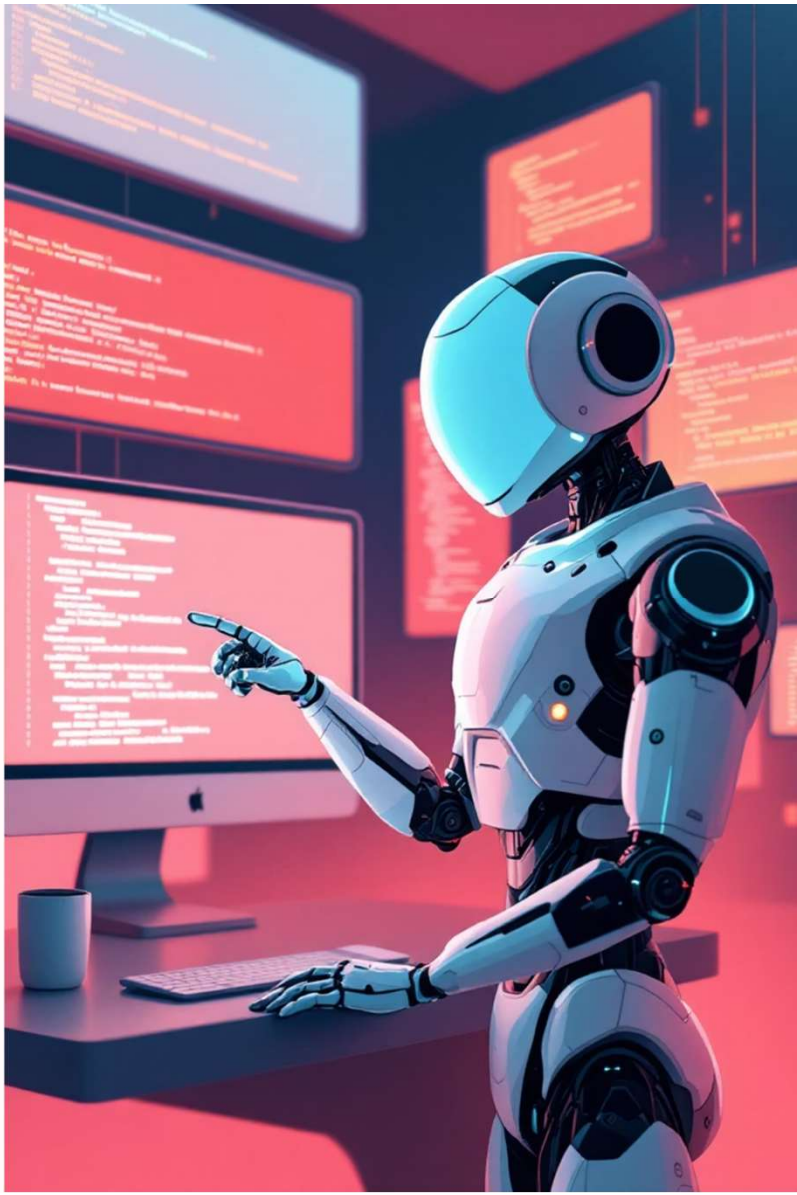
Automatic software development with AI

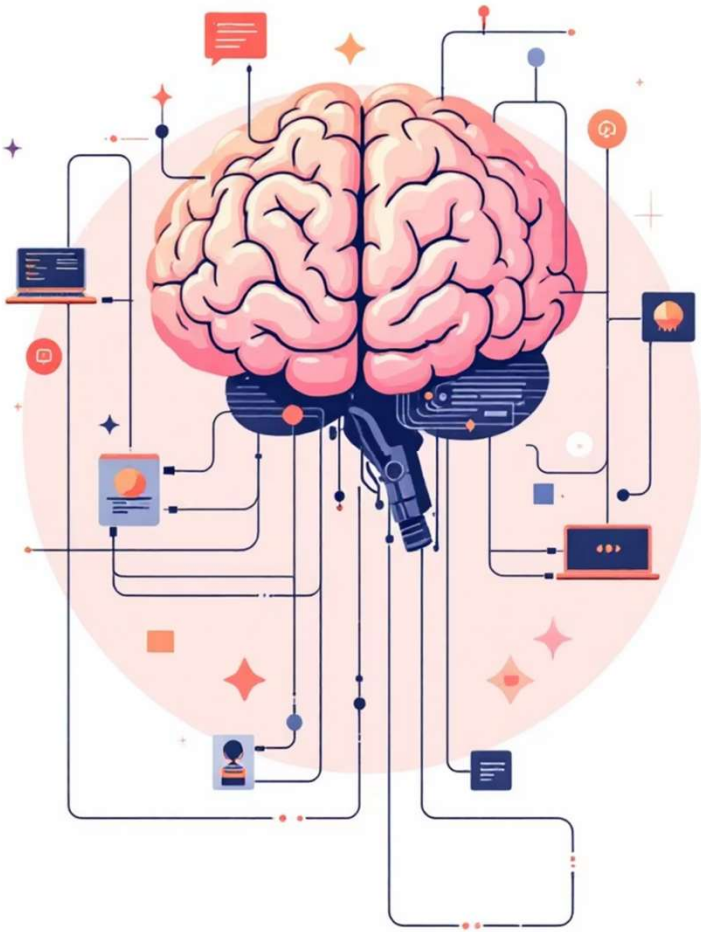
Project Guide
Mr. N. Venkateswara Rao
Assistant Professor, Dept of CSE(AIML)

Team Members
A. Sravani (Y22ACM406)
A. Sri Lakshmi(Y22ACM402)
A. Sai (Y22ACM405)
K. Venkata Krishna(Y22ACM447)
T. Nagendra (L23ACM501)



Abstract

An AI-powered coding assistant developed using Lang Graph automates the software development process through a multi-agent architecture. It converts natural language requirements into complete, functional software projects by simulating real-world developer workflows. The system employs specialized agents, including a Planner Agent to analyze requirements, an Architect Agent to design system structure, and a Coder Agent to generate executable code. By coordinating these agents, it ensures structured project creation, improved code consistency, and faster development. The platform is highly beneficial for students, developers, and rapid prototyping, promoting efficient and scalable software engineering practices.



Existing System – Lovable AI

Lovable AI is an AI-based development tool mainly focused on rapid UI creation and prototyping.
While it is useful for generating frontend components quickly, it faces several limitations when building complete software systems.

Single-Agent Approach
Limits complex reasoning and problem-solving.

UI-Focused Generation
Prioritizes UI over critical backend logic.

Isolated Code Snippets
Generates fragments, not cohesive projects.

Limited File Understanding
Struggles with project structure and dependencies.

Scaling Difficulties
Not suitable for large or complex applications.

Proposed System

A sophisticated multi-agent system is introduced, where each agent specializes in a critical development phase, mimicking a highly efficient human team. This modular approach ensures precision and speed in project execution.

Architecture Planner

Analyzes natural language requests to generate a detailed project plan.

Architect Agent

Breaks down plans into specific engineering tasks with clear context for each file.

Coder Agent

Implements each task directly into files, utilizing available tools like a human developer.

How our system Addresses These Challenges

Overcomes these limitations by adopting a **multi-agent development model**.

Solutions Provided:

- Uses **Planner, Architect, and Coder agents** for task separation
- Supports **end-to-end project generation** (frontend + backend)
- Generates **structured, multi-file codebases**
- Maintains clear **context awareness for each file**
- Scales effectively for **complex and real-world applications**

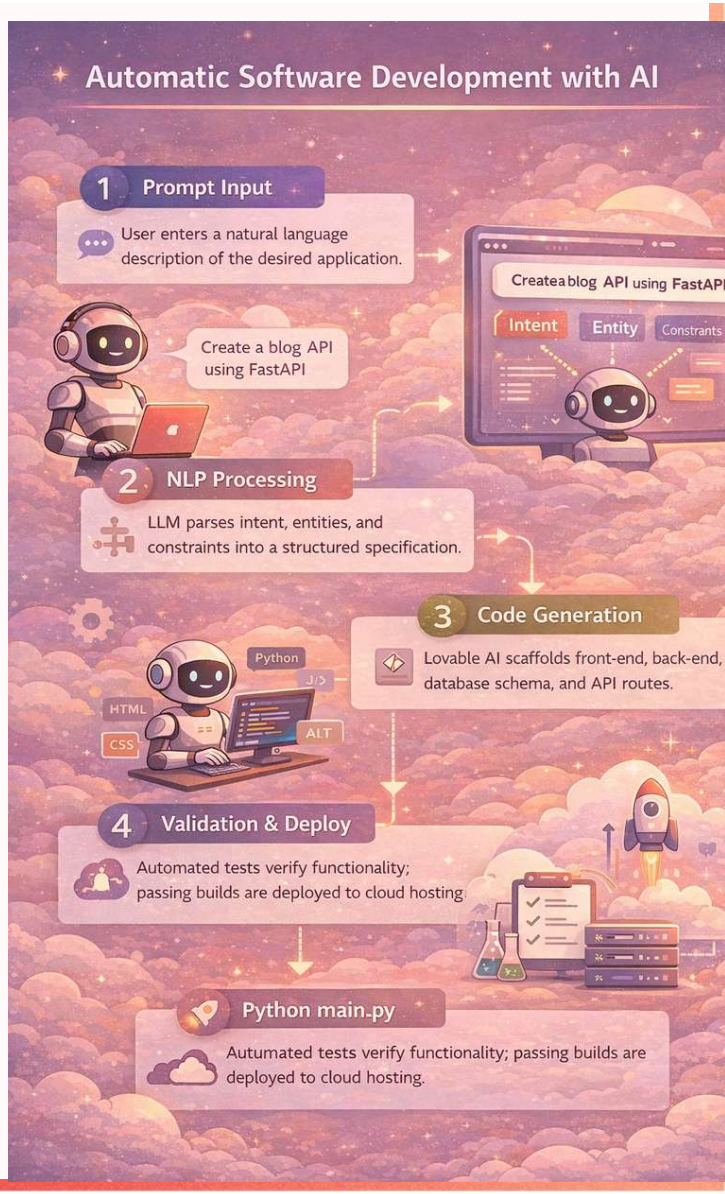
Design Methodology

1. **Prompt Input**: User enters a natural language description of the desired application.

2. NLP Processing: LLM parses intent, entities, and constraints into a structured specification.

3. Code Generation: Lovable AI scaffolds front-end, back-end, database schema, and API routes.

4. Validation & Deploy: Automated tests verify functionality; passing builds are deployed to cloud hosting.



System Implementation Overview

Lang Graph workflow: Input Processing → Multi-Agent Execution → Orchestration

Input Processing

User provides natural language prompt and passes it into the Lang Graph workflow.

01

Planner Agent

Analyzes requirements and generates a comprehensive project plan.

02

Architect Agent

Breaks plan into file-level tasks and defines structure & dependencies.

03

Coder Agent

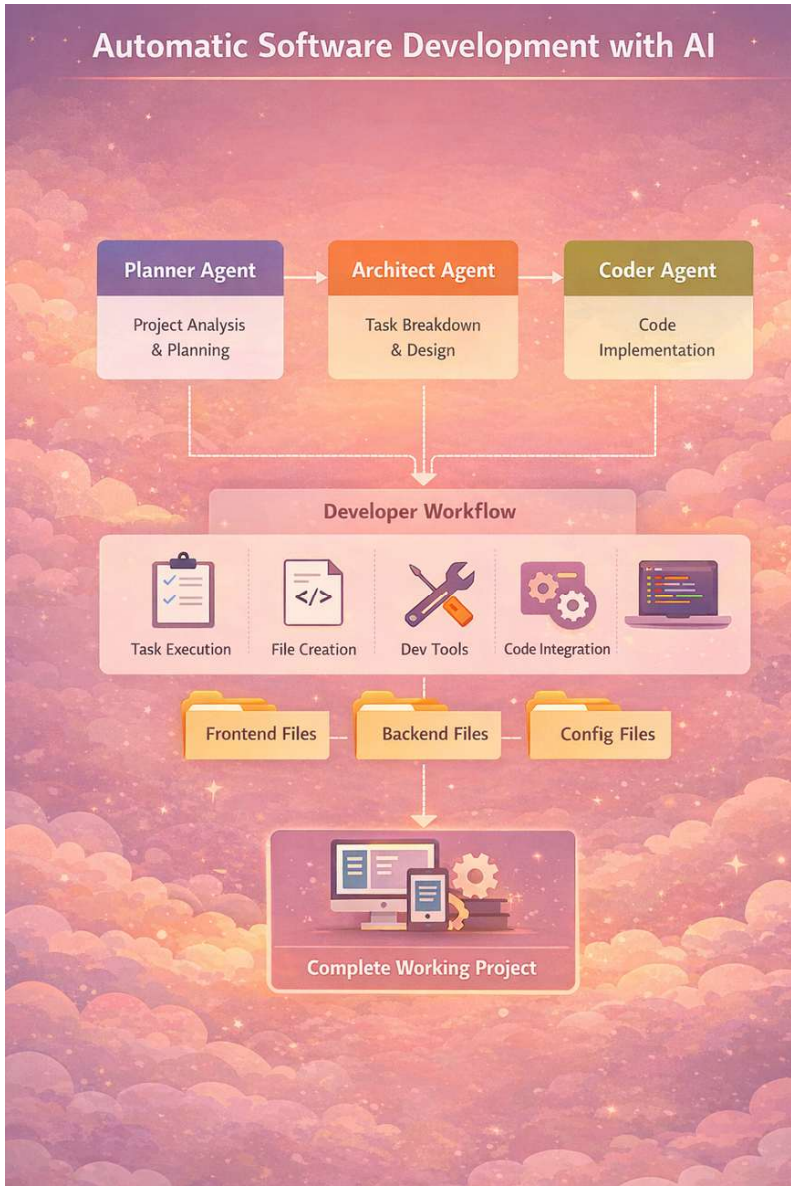
Writes actual code and creates files dynamically based on tasks.

04

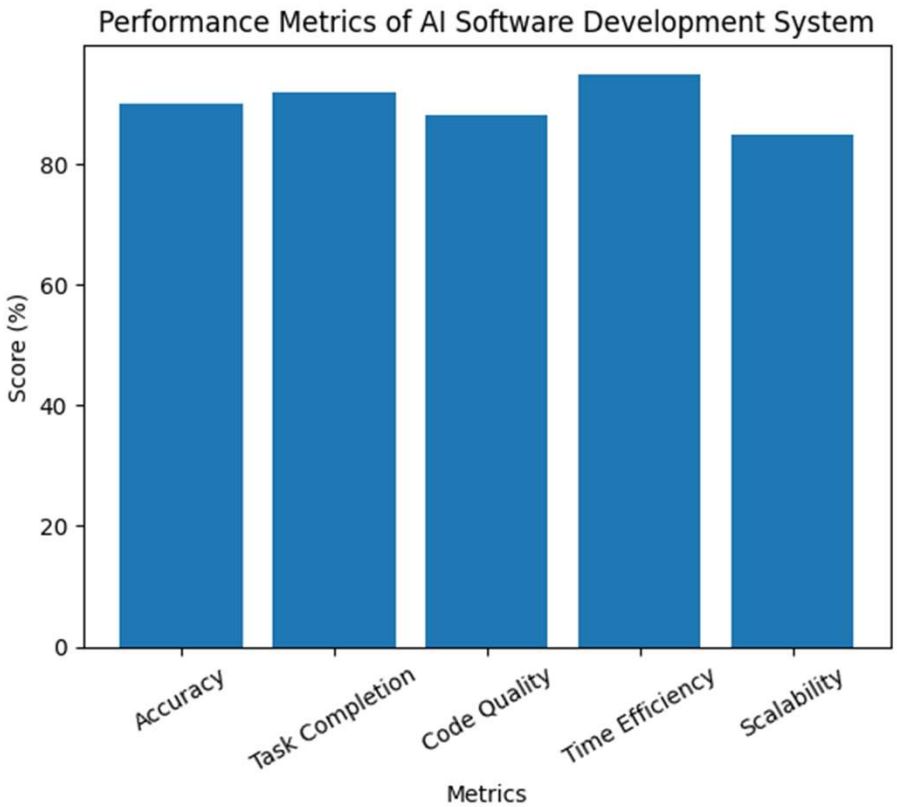
Orchestration

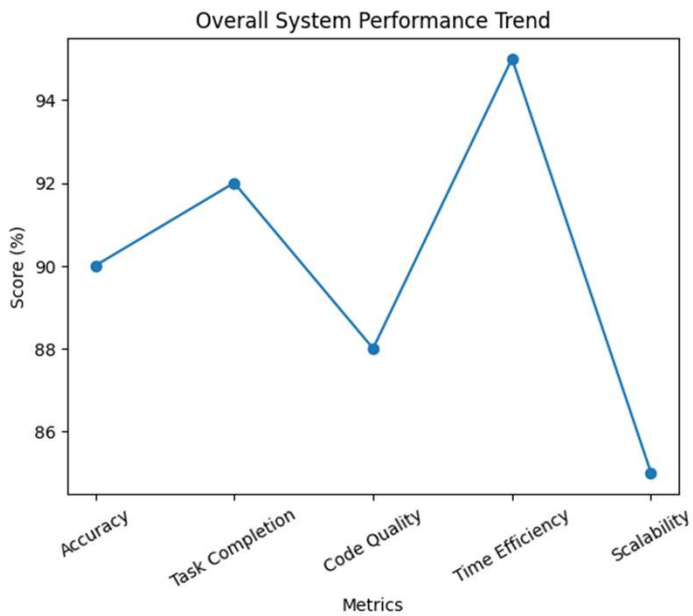
LangGraph manages sequential execution, context sharing, and error handling.

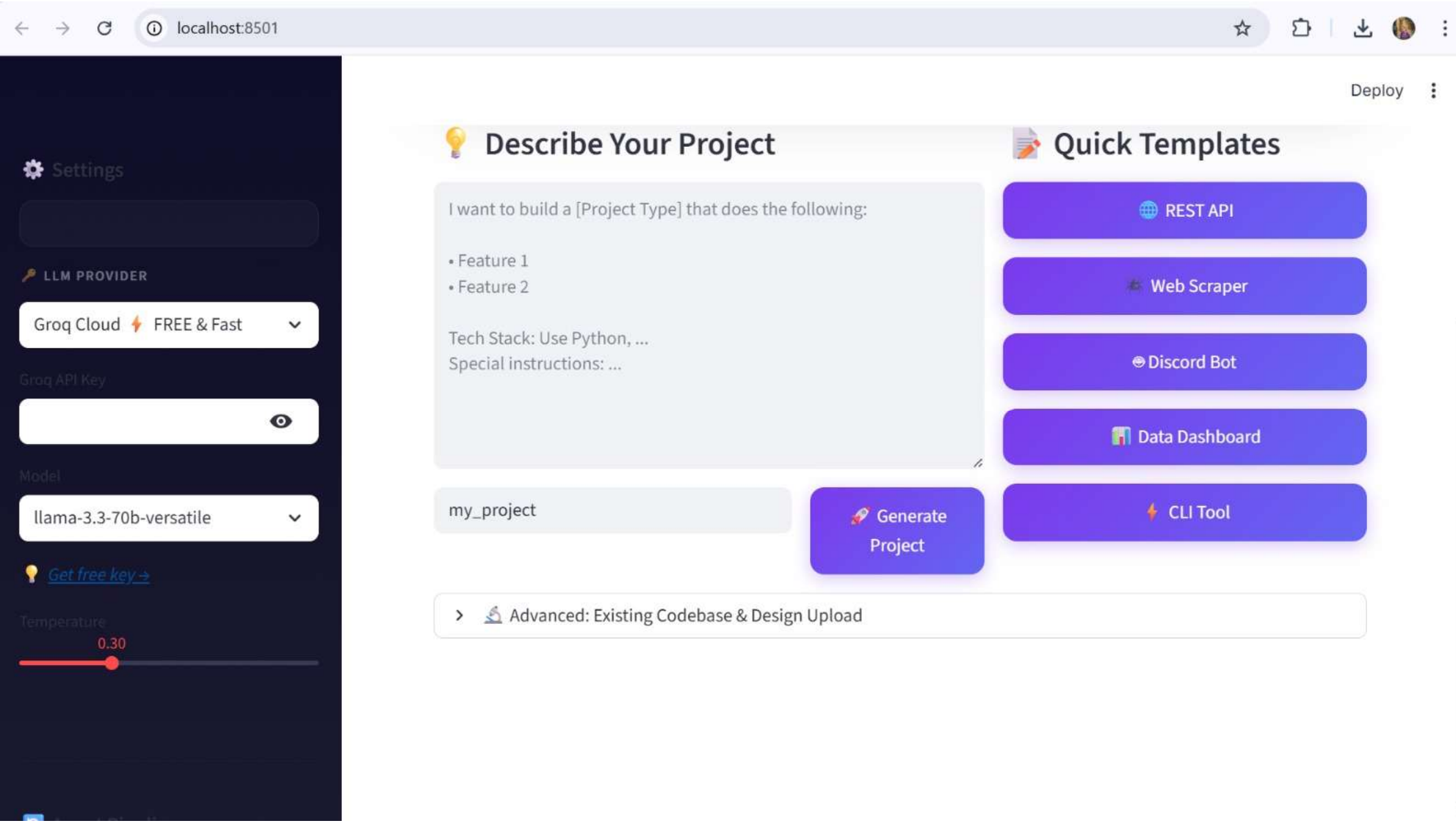
05


Automatic Software Development with AI
Planner Agent
Project Analysis & Planning
Architect Agent
Task Breakdown & Design
Coder Agent
Code Implementation
Developer Workflow
Task Execution
File Creation
Dev Tools
Code Integration
Frontend Files
Backend Files
Config Files
Complete Working Project

Results


Performance Metrics of AI Software Development System
Score (%)
0
20
40
60
80
Accuracy
Task Completion
Code Quality
Time Efficiency
Scalability
Metrics


Overall System Performance Trend
Score (%)
86
88
90
92
94
Accuracy
Task Completion
Code Quality
Time Efficiency
Scalability
Metrics


localhost:8501
Deploy
Settings
LLM PROVIDER
Groq Cloud FREE & Fast
Groq API Key
Model
llama-3.3-70b-versatile
Get free key →
Temperature
0.30
Describe Your Project
I want to build a [Project Type] that does the following:
• Feature 1
• Feature 2
Tech Stack: Use Python, ...
Special instructions: ...
my_project
Generate Project
Quick Templates
REST API
Web Scraper
Discord Bot
Data Dashboard
CLI Tool
Advanced: Existing Codebase & Design Upload

Settings

LLM PROVIDER

OpenRouter FRE...

OpenRouter API Key

Model

openrouter/free

RECOMMENDED — Free AI models!

`openrouter/free` auto-picks the best available model (avoids rate limits).

Get free key →

Temperature

0.30

Agent Pipeline

Stop Deploy

Describe Your Project

Build a Snake game web app with:
- Arrow key controls
- Score counter and high score
- Speed increases as score grows
- Game over screen with restart button
- Modern neon design on dark background
- Responsive canvas

Single index.html file with embedded CSS and JS.

snakegame

Generate Project

Quick Templates

Portfolio Website

Landing Page

REST API

Data Dashboard

CLI Tool

Advanced: Import Existing Project & Design Upload

Pipeline Running...

Architect Agent — Creating engineering tasks...

Architect complete!

Deploy

Settings

LLM PROVIDER

OpenRouter FRE...

OpenRouter API Key

Model

openrouter/free

RECOMMENDED — Free AI models!

`openrouter/free` auto-picks the best available model (avoids rate limits).

Get free key →

Temperature

0.30

Agent Pipeline

Git Integration (Updated)

Repository: `E:\Srilu_Project\generated_projects\build_a_snake` **Branch:** `master` **Commits added:** 1

Recent Commits

- `3273ba1` 09:42:13 — feat(build_a_snake): add frontend templates and components
- `67ea193` 01:45:39 — feat(build_a_snake): add frontend templates and components
- `60ba320` 01:45:39 — chore: add .gitignore

Project File Tree

```
└── generated_output.html
```

Download Project

Download ZIP | Save to Disk | Auto-saved to: `E:\Srilu_Project\generated_projects\my_project`

Start New Project | Download All Files (ZIP)

Settings

LLM PROVIDER

OpenRouter FRE...

OpenRouter API Key

Model

openrouter/free

RECOMMENDED — Free AI models!

`openrouter/free` auto-picks the best available model (avoids rate limits).

Get free key →

Temperature

0.30

Agent Pipeline

Deploy

Results

Overview | Code & Files | Live Preview | Project Plan | Architecture | Review & Revise | Sandbox | Git Log

Successfully generated 1 files!

Project Generation Complete!

Generated Files (1 files):

- `generated_output.html`

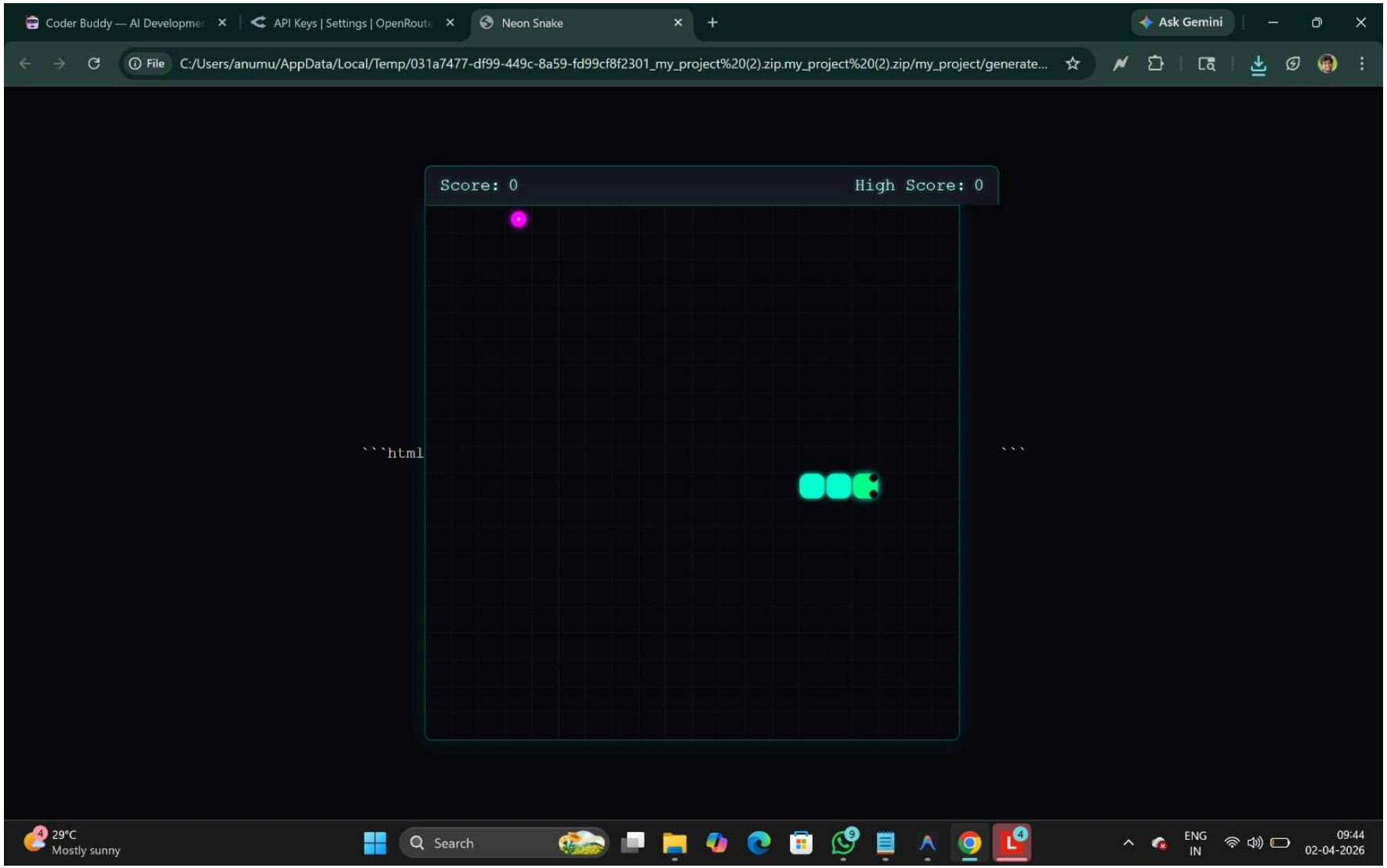
Review Report:

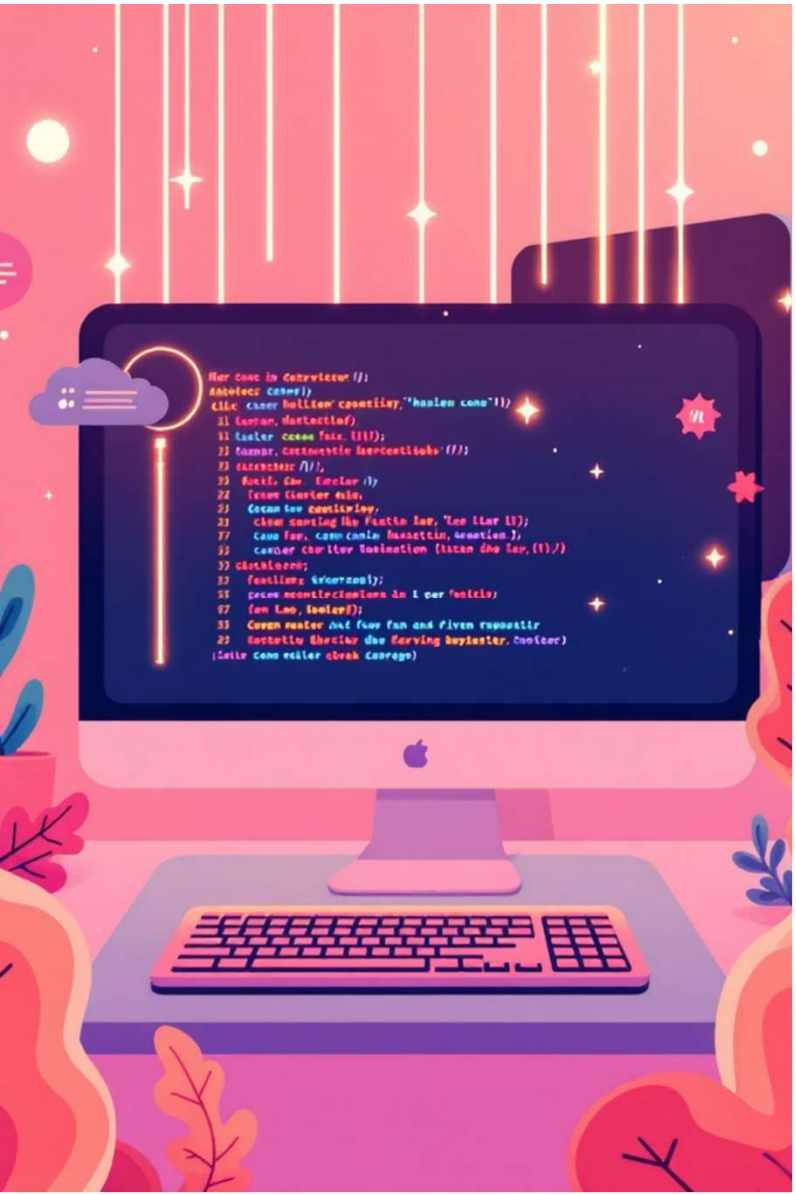
Review Summary

Overall Verdict: PASS **Quality Score**: 9/10

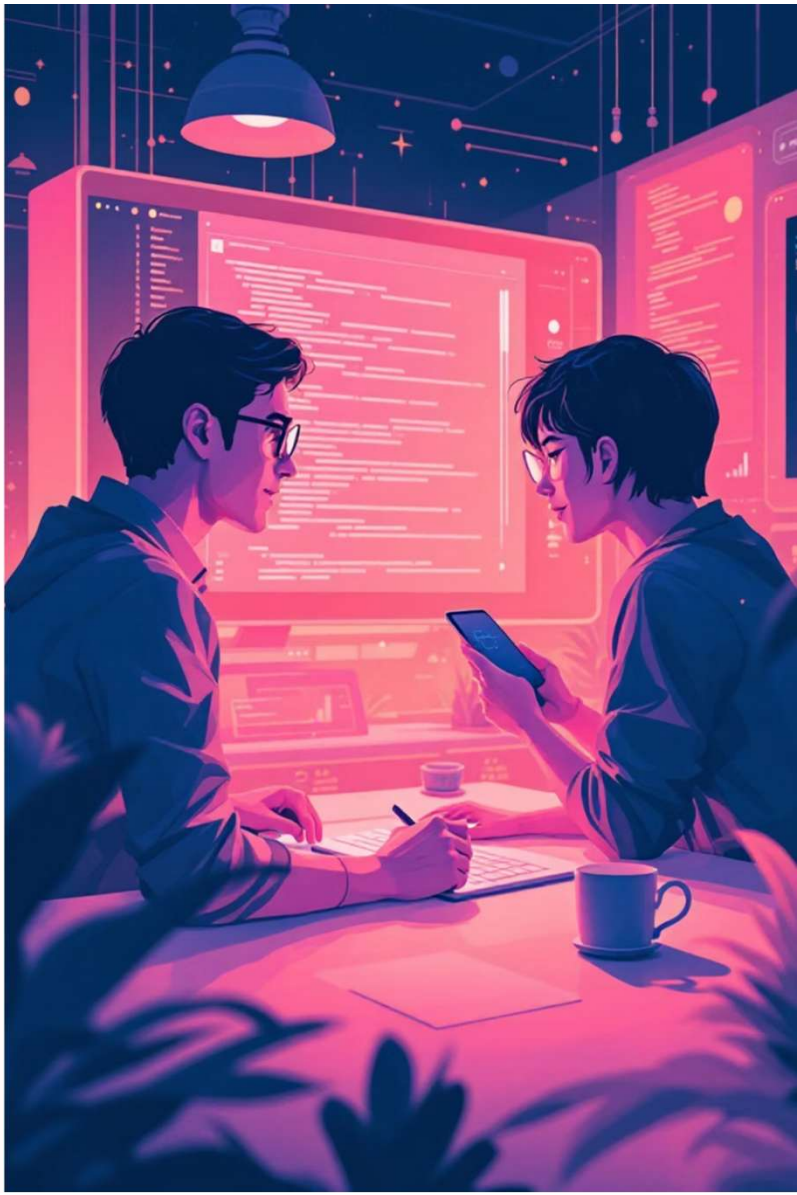
Issues Found

- **Severity**: Warning
- **File**: `index.html`
- **Issue**: The `growSnake()` method in the `GameEngine` class is defined but not used. The growth logic is handled directly in the `update()` method, which is functional but


Coder Buddy — AI Developmer
API Keys | Settings | OpenRoute
Neon Snake
Ask Gemini
File
C:/Users/anumu/AppData/Local/Temp/031a7477-df99-449c-8a59-fd99cf8f2301_my_project%20(2).zip.my_project%20(2).zip/my_project/generate...
Score: 0
High Score: 0
```html
```
29°C
Mostly sunny
Search
ENG
IN
09:44
02-04-2026



Conclusion

This system transforms traditional software development into a faster and more efficient workflow using Artificial Intelligence. It converts natural language prompts into complete and functional applications, significantly reducing manual coding effort. By utilizing a multi-agent architecture consisting of Planner, Architect, and Coder agents, it ensures a structured and systematic development process. The approach improves productivity, enhances accuracy, and makes software development more accessible to users with limited programming knowledge. Overall, it highlights the potential of Large Language Models in enabling intelligent, automated, and scalable software engineering solutions.

THANK YOU